



WELCOME – Unit 6

**IN403 – Deep Learning and Artificial
Intelligence**

Data Drift

- What model drift is
- The two main types of drift
- Why drift happens
- How to detect drift
- How to fix it
- Best practices for maintaining ML models

What is Data Drift

Definition: Model drift is the gradual decline in a machine learning model's performance because the real world changes over time.

Imagine you train a machine learning model that predicts whether an email is spam.

When you first deploy it, it achieves **98% accuracy**. Six months later, the accuracy drops to **82%**—even though you never changed the model.

This is called **model drift**.

The model learned patterns from historical data. When reality changes, those patterns may no longer be correct.

Why Does Model Drift Happen?

Machine learning models make assumptions based on their training data.

Examples of real-world changes include:

- New customer behavior
- Different weather patterns
- New products
- Updated sensors
- Changes in language
- New regulations
- Changes in business policies

If these changes weren't present during training, the model may make poor predictions.

How an ML System Works

A typical machine learning system has three parts:

Step 1: Feature Extraction

Raw information is converted into numbers.

Examples:

- Images to pixels
- Audio to sound waves
- Text to embeddings
- Sensor readings to numerical values



Step 2: The model learns relationships between inputs and outputs.

For example:

Image → Dog

Image → Cat



Step 3: Output

The prediction is converted into something useful.

Examples:

- Show a label
- Recommend a movie
- Approve a loan
- Detect fraud

2 main types of drifts

Type 1: Data Drift (or feature drift, or covariate drift)

What changes?

The input data changes, but the task stays the same.

Example

Suppose a self-driving car is trained using one camera. Later, the camera is replaced with a newer model.

The images now have:

- different colors
- different brightness
- different resolution

The world hasn't changed.

Only the data looks different.

The model may struggle because it has never seen these new inputs.



Another Example

A sentiment analysis model is trained on tweets from 2022.

By 2026, people use completely different slang.

The language changes.

The meaning stays similar.

This is data drift.

Detecting drift using mean and standard deviation

Suppose a machine learning model predicts house prices based on **house size (square feet)**.

Statistic	Training Data	New Data
Mean	2,000 sq ft	2,400 sq ft
Standard deviation	300 sq ft	320 sq ft

The increase in mean from **2,000** to **2,400** suggests that newer houses are generally larger than those seen during training.

This is evidence of data drift.



Mean unchanged, but standard deviation changes

Statistic	Training	New
Mean	100	100
Standard deviation	10	35

Although the average remains the same, the data are much more spread out.

A limitation of summary statistics

Summary statistics can completely miss some kinds of drift.



How to Fix Data Drift

Usually:

- Collect new data
- Label it
- Retrain the model

Often, no architecture changes are needed.

Type 2: Concept Drift

Concept drift is more serious.

What changes?

The **actual problem changes**.

The relationship between inputs and outputs is no longer valid.

Example: Bridge Toll System

Imagine an ML system that charges:

- Pedestrian → \$0.50
- Car → \$1.00

The model uses:

- object size
- speed

Everything works well, then the city allows bicycles on the bridge.

Policy: Bikes should pay the pedestrian price.

The model has never seen bicycles.

Because bikes are:

- larger than people
- faster than people

The model incorrectly classifies many bicycles as cars.

Nothing is wrong with the software.

The **rules of the real world changed.**



How to Fix Concept Drift

Simple retraining may not be enough.

You might need to:

- create new features
- redesign the model
- choose a different algorithm
- increase model complexity

Data Drift vs Concept Drift

Data Drift

Input data changes

Model stays mostly valid

Usually fixed by retraining

Easier to solve

Concept Drift

Problem definition changes

Model assumptions become invalid

Often requires redesign

Harder to solve

How to detect drifts

Method 1: Monitor Model Performance

Simply watch how well the model performs.

Examples:

- Accuracy
- Precision
- Recall
- Customer complaints
- Business metrics

Example:

A chatbot suddenly receives many negative reviews.
That may indicate drift.

Method 2: Monitor the Model Internally

Instead of waiting for failures, monitor what's happening inside the ML system.

Examples include:

Feature monitoring - Compare today's input data with training data.

Questions to ask:

- Are images darker?
- Are customers older?
- Has text vocabulary changed?

Large statistical differences may indicate data drift.

Title:

Confidence monitoring

Many classifiers output confidence scores.

Example:

Cat: 99%

Dog: 1%

High confidence is good.

If predictions become:

Cat: 42%

Dog: 40%

Rabbit: 18%

The model is becoming uncertain

This can be an early warning sign.

How to fix drifts

For Data Drift

Typical workflow:

- Collect new data
- Label the data
- Retrain the model
- Test performance
- Deploy the updated model

Often this is enough.

Title:

For Concept Drift

You may need to:

- redesign features
- add new data sources
- increase model complexity
- choose another ML architecture
- rethink the prediction task

This is a larger engineering effort.

Why Human Annotation Matters

Retraining requires fresh labeled data.

Human annotators help by:

- labeling new examples
- identifying edge cases
- maintaining consistent labels
- spotting changes in the real world

Without high-quality labels, retraining often produces poor models.

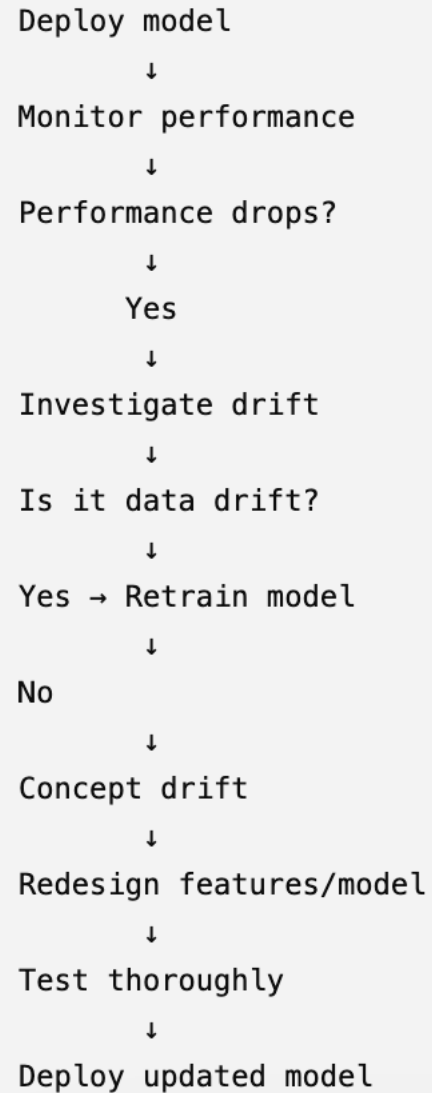
Best practices

Best Practices

To reduce the impact of drift:

- Define baseline performance when the model is first deployed.
- Continuously monitor accuracy and input data.
- Set alerts when performance drops.
- Regularly compare predictions with human-labeled examples.
- Keep versioned records of datasets and models.
- Build automated retraining pipelines.
- Involve domain experts when unusual patterns appear.
- Thoroughly test retrained models before deployment.

Example workflow



Summary

- **Model drift** is the decline in model performance as the real world changes.
- **Data drift** happens when the input data changes but the task remains the same. It is often fixed by retraining with updated data.
- **Concept drift** happens when the relationship between inputs and outputs changes. It usually requires changes to the model design or features.
- Detect drift by monitoring both **model performance** and **input data distributions**.
- Continuous monitoring, high-quality labeled data, and regular retraining are essential for keeping machine learning systems accurate over time.

- What MLOps is
- Why MLOps is important
- How Azure Machine Learning supports MLOps
- The lifecycle of a machine learning model
- How models are trained, tracked, deployed, and monitored
- Best practices for managing ML projects

What is MLOps

MLOps (Machine Learning Operations) is the practice of managing the entire lifecycle of machine learning models.

Think of it as **DevOps for Machine Learning**.

Instead of only building models, MLOps helps you:

- build models
- test them
- deploy them
- monitor them
- improve them over time

Without MLOps, ML projects become difficult to reproduce, maintain, and scale.

What do we need MLOps

Imagine a data scientist trains a model on their laptop.

Questions quickly arise:

- Which dataset was used?
- Which version of the code?
- Which Python libraries?
- Who trained the model?
- When was it deployed?
- Why did accuracy drop?

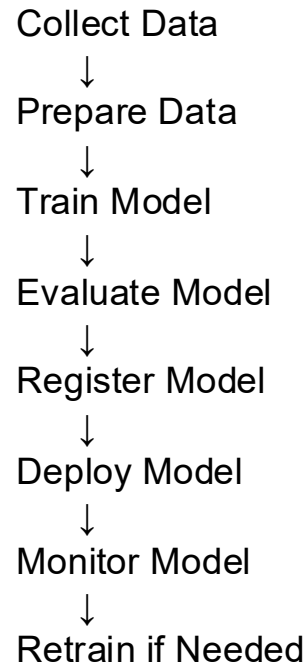
MLOps answers these questions by organizing and automating the ML workflow.

Benefits of MLOps

- Faster model development
- Faster deployment to production
- Better collaboration between teams
- Easier debugging
- Better reproducibility
- Automated retraining and deployment
- Continuous monitoring
- Full tracking of model history (lineage)

ML lifecycle

A typical ML project follows these stages:



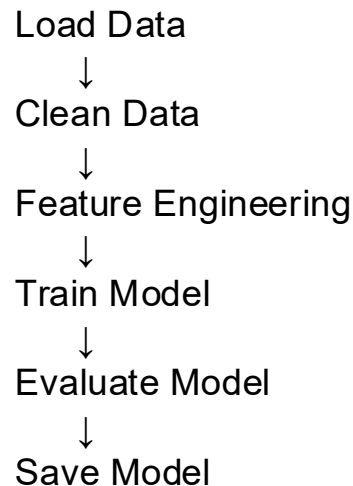
Azure Machine Learning provides tools for each step.

Reproducible ML Pipeline

An **ML pipeline** is a sequence of steps that automate model training.

Instead of running each step manually, Azure executes the entire workflow automatically.

Example:



Reusable Software Environments

Benefits

- Repeatable
- Easy to modify
- Saves time
- Reduces errors
- Supports collaboration

You can also clone an existing pipeline to experiment without changing the original.



A common problem in ML is:
"It works on my computer, but not on yours."

This usually happens because different machines have different software versions.

Azure Machine Learning solves this with **environments**.
An environment records:

- Python version
- pip packages
- Conda packages
- Dependencies

This ensures the same code runs consistently on any machine.

Model Registration

After training, you should **register** your model.

Think of a **model registry** as a library for trained models.

Instead of storing files randomly, models are organized with:

- Name
- Version
- Metadata
- Tags

```
CustomerChurn  
Version 1
```

```
CustomerChurn  
Version 2
```

```
CustomerChurn  
Version 3
```

Every time you improve the model, Azure automatically creates a new version.

Why Register Models?

- Easy to find
- Easy to deploy
- Supports version control
- Tracks model history

Metadata provides extra information about a model.

Examples include:

- Who trained it
- Training date
- Dataset used
- Accuracy
- Project name
- Deployment status

Example:

Project = FraudDetection

Department = Finance

Stage = Production

Tags make models easier to search and organize.

Metadata packaging

Before deployment, Azure packages the model into a **Docker container**.

A Docker container includes:

- Model files
- Python code
- Required libraries
- Runtime environment

This allows the model to run consistently across different computers and cloud services.

Deploying Models

Deployment makes a trained model available for real users or applications.

To deploy a model, Azure typically needs:

- the trained model
- a scoring (entry) script that processes requests and returns predictions
- the software environment (dependencies)
- deployment settings (CPU, GPU, memory, etc.)

For MLflow models, Azure can often infer the scoring script and environment automatically.

Endpoints

An **endpoint** is the URL or service through which applications send data to a deployed model.

Applications can call the endpoint to receive predictions.

2 types of endpoints

Type 1 - Online Endpoint
- used for real time predictions

Common use cases:

- Chatbots
- Fraud detection
- Recommendation systems
- Medical diagnosis
- Self-driving cars

Type 2 – Batch endpoint

- Used for processing **large amounts of data at once.**

Suitable for:

- Monthly reports
- Customer segmentation
- Bulk risk scoring

Safe Model Deployment (Controlled Rollout)

A/B Testing

Two versions run simultaneously.

Example:

Model A → 80% of users

Model B → 20% of users

You compare their performance before making Model B the default.

Traffic Splitting

Traffic can be adjusted gradually:

Week 1

Old Model → 90%

New Model → 10%

If the new model performs well:

Old Model → 30%

New Model → 70%

Eventually:

Old Model → 0%

New Model → 100%

This reduces deployment risk.

Monitoring Models

Deployment is not the end of the process.

Models must be monitored to ensure they continue performing well.

Azure can monitor:

- Prediction accuracy
- Input data changes (data drift)
- Infrastructure health
- CPU and memory usage
- Response time
- Deployment status

Monitoring helps detect problems early.

Model Lineage

Lineage is the complete history of a model.

It records:

- dataset used
- code version
- training job
- environment
- evaluation metrics
- deployment history
- person who trained the model

This is useful for debugging, auditing, and regulatory compliance.

Event Notifications

Azure can automatically generate notifications when important events occur.

Examples:

- Training completes
- Model registered
- Deployment succeeds
- Deployment fails
- Data drift detected

These events can trigger automated workflows or alerts.

Automation with CI/CD

CI/CD (Continuous Integration and Continuous Deployment) automates the ML lifecycle.

Benefits include:

- Faster releases
- Reduced manual work
- Consistent deployments
- Easier collaboration

Developer updates code

↓

Pushes to Git repository

↓

Azure Pipeline starts

↓

Model trains automatically

↓

Model evaluated

↓

Tests pass

↓

Model deployed automatically

Complete Azure MLOps Workflow

his continuous cycle
keeps models accurate
and up to date.

```
graph TD; A[Collect Data] --> B[Prepare Data]; B --> C[Create ML Pipeline]; C --> D[Train Model]; D --> E[Evaluate Model]; E --> F[Register Model]; F --> G[Package into Docker]; G --> H[Deploy Endpoint]; H --> I[Monitor Performance]; I --> J[Detect Drift]; J --> K[Retrain Model]; K --> L[Deploy New Version];
```

Collect Data
↓
Prepare Data
↓
Create ML Pipeline
↓
Train Model
↓
Evaluate Model
↓
Register Model
↓
Package into Docker
↓
Deploy Endpoint
↓
Monitor Performance
↓
Detect Drift
↓
Retrain Model
↓
Deploy New Version



To manage machine learning models effectively:

- Use pipelines to automate repetitive workflows.
- Use reusable environments to ensure consistent software dependencies.
- Register every trained model with version numbers.
- Add metadata and tags for organization and traceability.
- Package models into containers before deployment.
- Test models locally before deploying to the cloud.
- Use online endpoints for real-time predictions and batch endpoints for large-scale processing.
- Deploy new models gradually using A/B testing or traffic splitting.
- Continuously monitor model performance, infrastructure, and data drift.
- Automate training and deployment with Git and CI/CD pipelines.

Summary

- **MLOps** applies DevOps principles to machine learning, helping teams build, deploy, and maintain models efficiently.
- **Azure Machine Learning** supports the full ML lifecycle, from data preparation and training to deployment and monitoring.
- **Pipelines** automate repeatable workflows, while **environments** ensure consistent software setups.
- **Model registration** provides version control and tracking, and **Docker packaging** makes deployment portable.
- **Endpoints** expose models for real-time or batch predictions.
- **Monitoring, lineage, event notifications, and CI/CD automation** help keep models reliable, traceable, and continuously improving over time.

MLOps: Continuous delivery and automation pipelines in machine learning (Google Cloud)

- What MLOps is
- Why machine learning is different from traditional software development
- The meaning of CI, CD, and CT
- The three levels of MLOps maturity
- How an ML pipeline works
- Best practices for building production-ready ML systems



MLOps (Machine Learning Operations) is a set of practices that combines:

- Machine Learning
- DevOps
- Data Engineering

Its goal is to automate the entire machine learning lifecycle—from collecting data to deploying and monitoring models in production.

Without MLOps

Collect Data



Clean Data



Train Model



Test Model



Deploy



With MLOps

Data



Automatic Validation



Training Pipeline



Model Testing



Deployment



Monitoring



Automatic Retraining

Why Isn't Machine Learning Like Normal Software?

The code determines the behavior.

Machine learning is different.

Instead of writing rules, it learns rules from data.

Why Isn't Machine Learning Like Normal Software?

Now your application depends on:

- code
- data
- trained model
- infrastructure
- monitoring

If any of these change, the model's performance can change too. That's why ML systems require additional automation and validation compared with standard software.

What Problems Does MLOps Solve?

Example: Trains a fraud detection model.

It performs well today.

Three months later:

- customer behavior changes
- new fraud patterns appear
- data format changes

The model becomes less accurate.

This is called **model drift**.

Without MLOps:

- nobody notices
- predictions become poor
- business loses money

With MLOps:

- monitor model performance
- detect drift
- retrain automatically
- redeploy a better model

Understanding CI, CD, and CT

CI – Continuous Integration

CD – Continuous Delivery

CT – Continuous Training

Continuous Integration

In software engineering, CI means:

- merge code
- run tests
- verify everything still works

In machine learning, CI also includes:

- validating datasets
- checking schemas
- testing feature engineering
- testing models
- verifying training code

Example:

Developer pushes new preprocessing code.

Automatically:

```
Run Unit Tests
      ↓
Validate Dataset
      ↓
Train Sample Model
      ↓
Pass?
      ↓
Merge
```

Continuous Delivery

Continuous Delivery automatically deploys new versions.

Traditional software:

New Code



Deploy Website

Machine Learning:

New Training Pipeline



Train New Model



Deploy Prediction Service

The entire ML workflow—not just an application—is deployed.

CT — Continuous Training

CT is unique to machine learning.

It means:

Automatically retrain models whenever new data becomes available or predefined triggers occur.

Example:

Every Night

New Sales Data



Retrain Model



Evaluate Accuracy



Deploy Better Version

MLOps Maturity Levels

Level 0 — Manual Process

Everything is done manually.

Notebook



Train Model



Export Model



Deploy Manually

Characteristics:

- manual
- slow
- difficult to reproduce
- error-prone

Common tools:

- Jupyter Notebook
- Excel
- Python scripts

Suitable for:

- learning
- research
- prototypes

Level 1 — ML Pipeline Automation

Now the training pipeline is automated.

New Data



Validate



Train



Evaluate



Deploy

Characteristics:

- automatic retraining
- repeatable workflows
- reusable components
- metadata tracking
- continuous training

Suitable for:

- growing ML teams
- production systems

Level 2 — Full CI/CD Automation

Everything becomes automated.

Developer Pushes Code



CI Tests



Build Pipeline



Deploy Pipeline



Automatic Training



Model Registry



Deployment



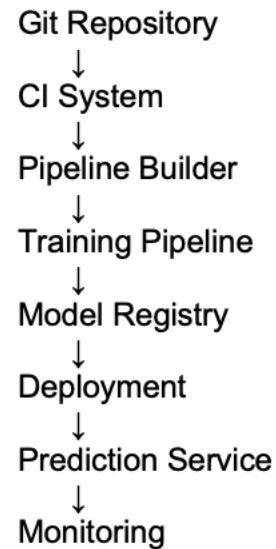
Monitoring

Characteristics:

- automated testing
- automated deployment
- versioned models
- scalable pipelines
- production-ready workflows

Components of a Modern ML Pipeline

A production ML pipeline commonly includes:



Each component has a specific responsibility.

Source Control

Stores:

- Python code
- notebooks
- configuration
- infrastructure definitions

Examples:

- Git
- GitHub
- GitLab

Data Validation

Before training begins:

Check:

- missing values
- schema changes
- unexpected categories
- duplicates
- outliers

Bad data produces bad models.

Training

The pipeline trains the model automatically.

Example:

Dataset
↓
Random Forest
↓
Model

Model Validation

Questions to ask:

Is the new model:

- more accurate?
- faster?
- less biased?
- stable?

If not:

Reject Deployment



Model Registry

A model registry stores model versions.

Example:

Version 1

Version 2

Version 3

This makes it easy to compare or roll back models.

Deployment

Deploy the approved model.

Possible targets:

- REST API
- batch inference
- mobile application
- cloud service

Monitoring

After deployment, monitor:

- latency
- prediction accuracy
- drift
- resource usage
- failures

Monitoring helps determine when retraining is needed.

8. Why Testing is Different in ML

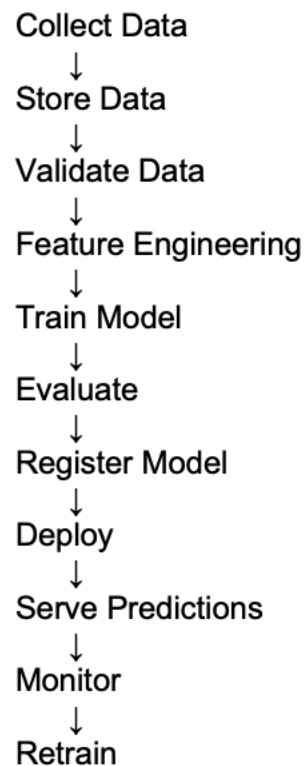
Software testing asks:

Does the code work?

Machine learning testing also asks:

- Is the data correct?
- Does training converge?
- Does the model produce expected outputs?
- Is accuracy acceptable?
- Are there NaN values?
- Do pipeline components integrate correctly?

A Complete Workflow



Once automated, this cycle can repeat with minimal manual intervention.

Key Takeaways

- **MLOps** applies DevOps practices to machine learning, adding automation, testing, deployment, and monitoring across the ML lifecycle.
- **CI (Continuous Integration)** validates not only code but also data, schemas, and models before changes are accepted.
- **CD (Continuous Delivery)** automates deployment of ML pipelines and prediction services, not just application code.
- **CT (Continuous Training)** automatically retrains models when new data or other triggers indicate it is needed.
- Organizations often progress through **three maturity levels**: manual workflows (Level 0), automated ML pipelines (Level 1), and fully automated CI/CD with continuous training (Level 2).

Next Step:

1. Learn Git and basic CI/CD concepts.
2. Build a simple ML model in Python.
3. Package your training code into reusable modules.
4. Experiment with pipeline orchestration tools such as Kubeflow Pipelines or Vertex AI Pipelines.
5. Add automated testing, validation, deployment, and monitoring to move from experimentation toward production-grade MLOps.